Euclid: A Parallel Programming first Game Engine

Oskar Niesen

March 2026

Abstract

Performance and networking are critical in designing many video games and simulations, however developers don't usually have the time to create and design an elaborate solution to these requirements. Developers rely on game engines to provide a foundation for them to build on and to speed up the process of development. Developers use parallel programming to drastically increase the performance of a simulation, however there are some areas such as memory management, starting threads early and non-determinism that make developing in a multi-threading environment difficult. Furthermore, many games require network synchronization which is a complex task since the simulations in different machines need to be kept in sync. Euclid aims to help developers by developing a performance by default game engine that helps developers write their program in such a way that simplifies the process of parallelization and networking.

1 Introduction

Game development is a major area of performance research, game developers want to have more detailed simulations without having to spend time optimizing and making sure the code is performant. The optimization of games takes more time and research, therefore lengthening the development process, therefore it is beneficial to rely on the underlying game engine to provide optimizations. This leads to the development of game engines that aim to expedite the game making process by reducing the amount of new work that needs to be done to the underlying simulation. This paper focuses on a new parallel programming paradigm for simulations that incorporate many objects and allow for some latency between instances of the simulation.

1.1 Problem

Games often want to use an increasingly larger number of objects for a more immersive simulation. However, processor clock frequency isn't scaling according to Moore's law anymore in order to keep up with the performance demands of the simulation. To get around this game developers need to use multi-threading to reach the performance that a real-time game demands. Multi-threading, however, isn't an easy topic and game developers need to spend a significant amount of time designing their game in a manner that is easy to parallelize.

Furthermore, game designers often want to incorporate multiplayer features into their design. This requires synchronization between each player's simulation in order to make sure that each player is viewing and interacting with the correct state of the game. Networking coupled with a large-scale simulation means that game developers can't send the location and state of every object over the network every frame. Therefore, it is more desirable to rely on an underlying simulation to keep the players and the server in sync. This means the underlying simulation must be deterministic, which imposes another important restriction on how multi-threading in the simulation must be handled. Determinism also helps in the area of debugging and testing by making the execution of the process repeatable. Unfortunately, enforcing determinism limits what can be parallelized

and has a performance cost, reducing the effects of parallelization [11].

To address the performance issue, game engine companies, such as Unity Technologies are developing a more data-oriented design pattern called the Entity-Component-System (ECS)[7]. ECS reduces the overhead of storing objects compared to a paradigm such as Object Oriented Programming (OOP). This is done by having a set of entities which each represent an object and each entity owns one or more components. Each component stores attributes or data that is assigned to the entity. Systems are used to act on all entities with certain components in order to mutate them and provide the program logic. Data oriented design organizes object data in arrays for efficient cache usage and helps reduce race conditions[1][10]. This project builds off of the design and ideas of ECS, separating data (entities and components) from the functions that mutate them (systems) in order to provide structure for parallelization.

1.2 Related Work

One of the biggest challenges when working with deterministic parallel code is how to handle shared memory in a safe and predictable way. Shared memory parallelism is usually split into two categories: strong determinism and weak determinism. Strong determinism ensures that the shared memory access is deterministic and weak determinism ensures that locks are acquired deterministically. The kendo algorithm enforces weak determinism by keeping track of a logical counter of how many locks a thread has encountered in order to make sure that each thread waits until its turn [5]. DThreads is an example that uses a hybrid shared memory system. It moves threads to their own process in order to isolate writes and prevent read-write conflicts. Similar to our work the output of each process the data written is then copied back to the main process during a serial phase in a deterministic manner [4]. Other research like Tapir focus on optimizing recursive fork-join parallelization [6].

1.3 Purpose

The current study proposes a new game engine, Euclid, that provides a simpler platform for developing a game with multi-threading, networking capabilities while maintaining a underlying deterministic simulation to the game developers. Euclid is a prototype for a deterministic multi-threading model that tries to encapsulate the difficulties of multi-threading from game developers. Euclid provides a system of multi-threading that is mainly lock-free and allows game developers to create multi-threaded behavior without having to worry about determinism, edge case race conditions, or synchronizing the simulation between clients and the server.

1.4 Limitations

There are some limitations of this design of Euclid that make its benefits possible and simplify the work done by the game developers. The biggest limitation is a high latency between user input and their effect actually taking place. Latency adds difficulty for game developers by requiring that player input must be verified to make sure that it is still valid. One way to mitigate latency this limitation is by having client owned objects that need to be very responsive to player input. For example, if a player is driving a car the player requires their inputs to have very fast response times. Game developers usually handle this by making the client own the object instead of the server. However, a compromise must be reached in order to keep the deterministic nature of the simulation, making the situation more challenging.

Another important limitation for some use cases is that it will be possible for players to cheat. The foundation of Euclid requires each client to run the entire simulation, and therefore must give each client access to information that the player might not have. This inevitability allows players to cheat by looking at the information that isn't displayed on the user interface. Cheating is not an important topic for all games and simulations.

In order to limit the scope of Euclid and focus on determinism, these limitations will be left for future research. This makes Euclid less suitable for compet-

itive games that require fast responsiveness to user input.

2 Game Engine Design

The Euclid game engine aims to simplify game creation. There are two major parts of a game that make game development complex for game developers. One is the requirement to synchronize the game between clients and the other is optimizing the game logic through the use of multi-threading. In order to simplify this problem the task of networking usually falls upon the underlying game engine. Until now there has been little that game engines could do to design a way to simplify the parallelization process for programmers.

2.1 Networking

The foundation of Euclid synchronizes by running a complete simulation of the game on the clients and server. This reduces the amount of synchronization required since each step in the simulation can be reached by each client, as only the starting point needs to be synchronized and they running the same program. A step in a simulation represents a fixed period of time in which the simulation logic deterministically modifies the state of the simulation. In this model the server is the source of truth and validates all outside modifications to the simulation before they are send to each client. Any time a client wants perform do an action on the simulation, it will need to send a message to the server first before the server will send the action back to the client and the action will be reflected in the simulation. See 1 for a visualization of how steps are synchronized.

2.1.1 Client-Server Synchronization

Running the simulation across multiple computers requires the underlying simulation to be deterministic, meaning that multiple copies of a simulation must reach the same state when run for a certain amount of steps. Under this model we need to make sure that each client can't reach a state that the server hasn't reached yet. As long as the client is in a state that the server has previously been in, the client can always continue the simulation in order to reach the current state of the server and be up to date again. This means the underlying simulation must have seeded randomness, deterministic multi-threading, and player input must be consistently applied to all simulations at the same step. However, this does mean that the time each step of the simulation is run does not have to be synced between the server and clients, therefore the clients are allowed to lag behind the server, at the cost of input latency.

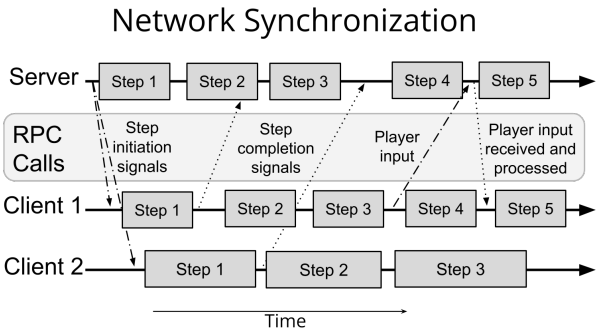

Figure 1: A visual representation of how the clients and server are synchronized through steps, RPC calls and player input.

2.1.2 Player Input

Most video games use player input to change the way the simulation behaves. This means that every simulation needs to apply the effects of player input in the same way in order to maintain consistency across all clients. Player input will be sent to the server via a remote procedure call (RPC) in order to apply the correct action. If the RPC is still valid, the simulation has already been updated and the server sends the RPC to each client to be run before they run this step of the simulation. This means that the clients may never be ahead of the server in the simulation or else they might miss some player originated input and they can't reverse the simulation. By ensuring the RPC is applied before the step in the simulation is

run we can ensure that each client has the same input to run the each step deterministically and therefore maintain the deterministic nature of the simulation.

Player input will not be applied to the simulation on the step that the player sees, instead it will be applied on the server's next step once the RPC is received. This means that there can be a high player input latency since the player will only see the update after the server sends the RPC back to the client. The input latency depends on how far behind the client is compared to the server. In some cases the server can speed up or slow down the rate of steps it takes in the simulation to balance input latency. This may make Euclid incompatible with some games that need to be very responsive, however there are many other simulations and game types where this could be very helpful.

2.1.3 Remote Procedure Calls

In order to simplify game development, we have created a special RPC model to handle network synchronization. When making an RPC, developers will bind a function along with the name of the RPC to the RPC manager, they may then call the RPC as a result of player input in a serial phase of the program. Each RPC has two parts that the developer needs to implement, the verification section and the actual call.

Every RPC is first handled by the server. Once the RPC arrives, the server processes it before the next simulation step. Each RPC contains two parts that the developer defines, the first part checks if the RPC is still valid and if it is valid the second applies the effects of the RPC, modifying the simulation. An RPC may be invalid for a few reasons. The first possibility is that the action might be illegal, meaning the player input should be dropped and the simulation should not be changed. The second is that the RPC may no longer be valid, as explained before, the server may receive the RPC much later than when the player sends it and the state of the simulation the it could be no longer valid. In this case the player input needs to be dropped and the player will experience a break in user experience. Lastly, due to the potential of high latency, the meaning of a valid RPC might change since objects in the game might have moved around. In this case the game developers will need to adapt the effect of the RPC to the new state. It is up to the game developer to return the invalid result before modifying the simulation in the case that the RPC is invalid.

If the RPC is valid then it will return the valid result and the server will then send the RPC to each client. The clients will then wait to process that step until they have received and applied all RPCs that the server verified before that specific step. This helps maintain the deterministic nature of the simulation. The clients won't have to verify the RPC before applying its effects because it is guaranteed that they are in the same state as when the server processed it.

2.2 Enity-Component-System

Euclid needs to organize the game data that developers allocate and modify in a way that is efficient and gives Euclid enough information to help out with multi-threading. The Entity-Component-System is a data-oriented design that organizes game objects into three different parts, entities, components and systems. Each entity has a runtime type based on what components it has and are stored in arrays based on their type along with the data of their components. Therefore each entity of the same type is the same size in bytes, making indexing and deletion quick in an entity array and easy. Each entity type also tells how to access a specific component that the entity holds using the address offset of the entity and a static_cast. Euclid supports dynamic entity types, when a component is added or removed from an entity it moves from its current entity type array and into the new one. This is also how Unity's ECS works, it helps that entities will rarely have their components added or removed [8]. During the application, developers can schedule systems to be applied to specific entity types. This registers a function that takes in an entity of that specific type for the game engine to call in parallel during the simulation.

2.2.1 UI Objects

Traditionally game engines have used an object-oriented approach to developing games, this allows the game logic associated with a game object to be very close to the UI logic of the same element. One issue with this is that the UI logic and the game logic tend to be tightly coupled together and can't be separated from each-other. This can be an issue since the server doesn't actually need to render any UI elements. Another reason why this might be undesirable is that it is harder and slower to run tests on the game logic when it is so tightly coupled to the UI. Decoupling the game logic an UI allows for Euclid to run a true simulation that doesn't need to be visualized. It allows for smoother animations through interpolation when the UI needs to update more often than the simulation. The UI is treated as a representation and animation of the simulation rather than the source of truth.

While decoupling the UI and game logic is beneficial, it does complicate the ECS implementation. Developers need to register a UI object type to each entity type that needs to be visualized. The ECS system then creates a new UI object of that type after an entity is created. The UI object is allowed to look at all of the data of the entity it is associated with, however the entity doesn't know that the UI object exists. Unlike the entity itself the UI object uses an object-oriented approach because it isn't parallelizable and a data-oriented design doesn't provide a performance benefit here.

2.3 Multi-Threading

There are a few requirements for our multi-threading design that come from the functional requirements of the game. First the multi-threading design needs to provide a performance boost over the serial design, otherwise there would be no reason to use multi-threading as it would unnecessarily complicate the game. Second, the design needs to be deterministic in order to maintain client-server synchronization. Finally the game developers need to find the implementation convenient and easy to use, otherwise it would be easier to implement their own parallelized code.

The simulation needs to be run on a CPU because GPUs can't perform high level game logic as it may access any object at random and have many condition statements. This design assumes that the program will run on at least 30 threads to justify the use of parallelization.

2.3.1 Enforcing determinism

There has been a lot of previous research trying to ensure deterministic behavior in a multi-threaded environment. The main reason for this is that deterministic behavior is important for reproducing bugs but it is also necessary to keep the remote clients synchronized [11].

Deterministic multi-threaded programs often run up to 8.4 times slower than regular non-deterministic programs [4]. One system of multi-threading is called DThreads has investigated holding thread private modifications until the thread is complete where it will then be merged deterministically in a serial phase. Each thread in DThreads creates a new process in order to keep a private copy of the original data. DThreads has some major limitations with regard to game development, it resolves merging results from by overwriting previous data which might have been changed by another thread.

An example of this is when two separate projectiles each have their own thread and are colliding with a target in the same parallel section. They both want to do damage to that target. Each thread will want to write the previous health of the target minus the projectile damage to the targets health in the serial section. Given that each thread only sees the original health, one thread will end up overwriting the other. While this still occurs deterministically, from a game perspective it reached an undesirable outcome. If the simulation would have been slightly different the projectiles might collide in a different step of the simulation and therefore a different parallel section. In this case they would both do their damage as normal because one would not be overwriting the other.

Our method utilizes these serial and parallel sections like DThreads. A block is a parallel section with one or more systems scheduled, each step is composed

of one or more blocks shown in figure 2. The systems within a block apply a series of identical function calls that run on a specific group of entities that have specific components. A function call in a system has one primary entity that it is related to and components that it is associated with and can modify any other component or entity. No function call in a system may rely on the output of anything within the same block and all must interact with each other system in its block deterministically. Each function call in the block is run in parallel until they are all completed at the end synchronization barrier at the end of the block. See figure 3 for a visual representation of systems and blocks. After the synchronization barrier of each block is a serial section where the main thread continues on in the step.

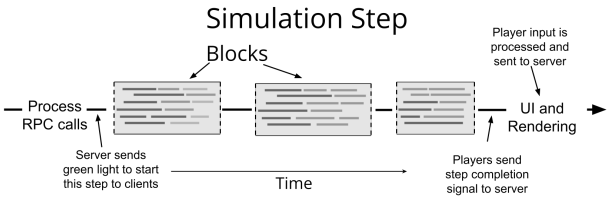

Figure 2: A visual representation of a simulation step. Each step contains multiple blocks and handles RPC calls before and after.

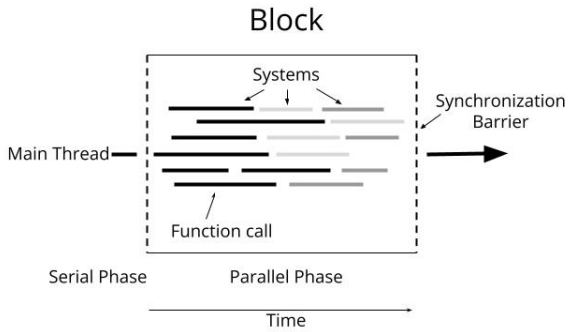

Figure 3: A visual representation of how a block is set up. Each system is a different shade of grey.

Developers control which systems can be scheduled in the same block together and need to insure that there is no data dependency between or within each system and that they do not overwrite each other in a logic breaking way. Scheduling systems in less blocks reduces the amount of barriers present and reduces the overhead when entering and exiting the parallel sections. Each barrier at the end of a block reduces the overall efficiency of multi-threading, therefore reducing the number of blocks within a step of the simulation will improve the simulation performance. In order to increasing the number of systems that can be within a block block we need to increase the compatibility of non-conflicting systems that can be in a block, reducing the number of blocks and subsequently increase the simulation performance.

2.3.2 Intra-block conflicts

Each function call in every system has the potential to read and write from anywhere in memory, allowing for a plethora of read/write and write/write conflicts that may cause nondeterministic behavior and race conditions. While locks can be used to prevent some race conditions they add a lot of overhead and don't enforce deterministic behavior, therefore the should be avoided when possible. DThreads handles this by writing each threads output to separate places and merges them together by assigning an order to the threads and having the last thread's output overwrite the other thread's output. As stated before this limits the usefulness of processing in parallel if the results from one thread are overwritten. Euclid focuses on improving the merging processes to correctly merge numerical values together and therefore prevent one thread from overwriting another.

Our implementation will use a public read-value and a set of public write-values for each numerical variable in every component. Public in this case means that every thread will have access to these values in memory. This removes any read/write conflicts since each thread will only write to the write-value of each variable and read from the read-value. After the block is finished we will do some work to update the read-value based on the contents of the write-values and reset the write-values. Therefore, as long

as the write-values are modified deterministically we can ensure the determinacy of each block.

First we start by identifying that there are six main operations on a numerical variable that we will be able to apply deterministically: addition, subtraction, multiplication, division, exponentiation and assignment. Our goal is to order a given set of these operations into a sequence in such a way that will always reach a deterministic result.

We find that there are 4 groups of associativity from these 6 operations, addition and subtraction, multiplication and division, exponentiation, and assignment. Any order of addition and subtraction operations, for example, will always lead to the same result since they are associative with each other, the same with multiplication and division. Each of these groups will store their intermediate value in a variable, instead of one per operation to reduce the number of intermediate values we need to store.

Now we give each group precedence to determine the order they are applied to the new write value. First, if there was any value assignment operation we use the stored value as our new read value. In the case where multiple assignment operations occur to the same location in one block, we use the id of the component that the thread is associated with to determine precedence. In the case that there is no assignment operation the ending value can be given by the following equation:

$$read_value^{exponentiation} \times \frac{multiplication}{division} + \frac{addition}{subtraction}$$

This calculation can be done at the time of the last write, during the following serial phase, or when first being read or written in a subsequent block. Ideally we would like to do any work possible in the parallel blocks, so updating the new value lazily in a later block might be best.

2.3.3 Scheduling systems into blocks

Developers will schedule systems and designate conflicting systems. The developer resolves these conflicts by telling the scheduler that one system has to go after another system. The scheduler will then create a topological graph and schedule each system in a block to be executed during the simulation step.

Amdahls law states that the speedup of a program given the parallel fraction f and the number of processors n is written as $Speedup(f, n) = \frac{1}{(1-f)+f/n}$ where $1 - f$ is the time of the serial parts of the program and f/n is the time of the parallel parts. Applying this to our block synchronization system, this limits the number of synchronization steps we can have, and therefore limits how many blocks can be used. If we want to run more steps in a simulation in the same period of time then the number of blocks will increase and therefore $1 - f$ will be the main limiting factor of performance [3]. Gustafson argues, however, that we can use an increase in parallelism to increase the problem size or resolution instead of increasing the speed of the simulation [2]. This means have larger and more accurate simulations rather than making the same simulation run faster. This is applicable to real time games where increasing the details of the game is more important than allowing the game logic to run quicker than the GPU needs to render. Euclid works to maximize this, as the problem size is increased the amount of work within each block is increased, but not necessarily the number of blocks.

3 Implementation

Euclid is written in C++ and the source code can be found on GitHub at https://github.com/tuvus/Euclid. It is designed to be fast enough not to incur overhead from the language but with enough features to simplify development. This is similar to most game engines which are also written in C++, such as Unreal and Unity. The goal of this project is not necessarily to make a *good* game engine and a game but rather to investigate a new multi-threading and networking paradigm.

3.1 Window and setup

The first goal of our project was to get a window set up along with a simple UI. We started off choosing

OpenGL as our rendering system along with GLFW3 and imgui. We decided to switch from imgui to Raylib which still uses OpenGL but allow us to add UI components easier.

3.2 Networking

After we created the basic UI the next goal was to get a server-client connection going. A client can either host a game and act as the server or connect to the server. To allow for maximum flexibility we decided to use the GameNetworkingSockets client-server API which is made by Steam. Sending data over the socket is primitive but also versatile. After setting up the networking we created a simple menu in imgui to allow the client to start hosting or connect to a server see figure 5 below.

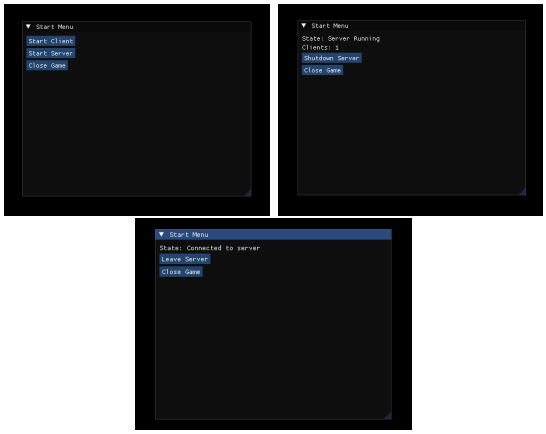

Figure 4: A simple client-server UI.

3.3 Remote Procedure Calls

The game engine design requires custom, easy to use remote procedure calls so that game developers can create their own RPCs, register them, and call them from the UI thread. Unfortunately this is an area in which we had the least experience in, which means that it was the biggest risk in the design.

The first package that we looked at that avoided the code generation step was ZPP_Bits. It is likely the quickest option out there as it does most of the RPC schema work at compile time using structs instead of a dedicated schema file. The problem that we encountered with ZPP_Bits was that it wasn't possible to wrap each RPC with logic from the game engine. We also realized that while ZPP_Bits is open source, its use of constexpr and template classes make the code quite difficult to understand and therefore unextendable.

Determining that ZPP_Bits was not an option we looked further into Protobuf, Flatbuffers and Capt'n Proto, each of these requiring a schema to make a RPC along with an extra code generation step before compilation in order to define how to call the RPC. This is not an ideal solution, because it doesn't allow for the game engine to wrap the RPC defined by the developers with special logic that determines where it should be run. This is necessary for RPC validation on the server and sending a valid RPC to the clients.

After further investigation we found another library, RpcLib, which allowed the game engine to treat RPCs as simple serialized function calls. We believe that this is a slower implementation in terms of performance but is very customizable. RpcLib focuses on a simple uni-directional RPCs that are sent from the server to clients, causing an issue because Euclid needs bi-directional RPCs and uses GameNetworkingSockets to handle networking. Therefore it was necessary to make a custom wrapper around RpcLib in order for the RPCs to work correctly. The result is that game developers can register different RPCs dynamically while the game is running and call them with parameters. The details of the RPC can be found in 2.1.3.

3.4 Entity Component System

Once we had a solid foundation for the networking and game we could move over to an Entity Component System (ECS) model. To switch over to ECS we needed to replace the old game manager system and decompose each game object into multiple entities, components and systems. Due to the parallel nature of the game logic, entities can't be added or removed from an entity array within a block without adding in a plethora of read/write conflicts. Therefore these add and remove commands are done after

the block. When an entity function from within the block wants do add or delete an entity that command along with the current entities ID is stored in a list to be processed after the block. In the serial section after the block the list is sorted stably in order to maintain deterministic behavior. Essentially the list is sorted first by the entity ID that made that command and by the time that it was added to the list so commands from the same entity are applied in order. Entity creation, however, is done in the parallel section right when the entity function creates the new entity. The new entity is added to a special list in the entity array which holds on to it until it can be added to the main entity array after the block. This ensures that the size of the entity array does not change during a block and that the most expensive operation, entity creation, is done in parallel.

3.5 Multithreading

Euclid doesn't create threads when needed, instead it creates a pool of threads called workers based on the amount of logical processors that the CPU provides. These workers pick up jobs from a serial queue that are scheduled at the beginning of a block and execute them. Each job contains one system along with a range of entities of which the worker should apply the function within the system to. This allows the entity functions to be efficiently batched and reduces the overhead of taking jobs from the queue. Euclid's job system is inspired by Unity's job system that also pre-allocates its worker threads [9].

3.6 Demo Game

Euclid's main focus is to investigate the proposed parallel programming paradigm, and, in order to test it we made a demo game that would utilize it's multi-threading and multiplayer capabilities. The game is designed to be a multiplayer, symmetrical, two team tower defense card game, where players play cards to place towers, send units and try to defeat the opponent. We chose this design because it is popular in the game industry and can make excellent use of multi-threading.

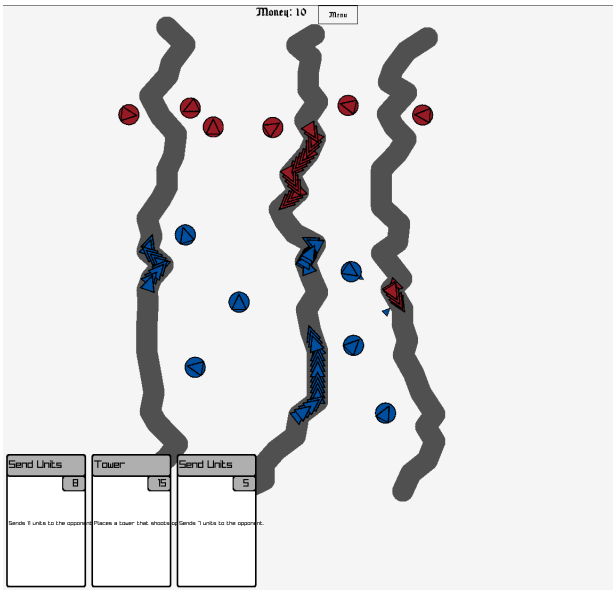

Figure 5: A tower defence game to demonstrate Euclid's multi-threading capabilities.

3.7 Future work

The next area of development is to optimize the multi-threading system which was discussed in 2.3.2. Further research should focus on efficiently merging the results is also important and may be the deciding factor on how effective the design is.

References

[1] Jessica D Bayliss. "The data-oriented design process for game development". In: *Computer* 55.5 (2022), pp. 31–38. DOI: https://doi.org/10.1109/mc.2022.3155108.

[2] John L. Gustafson. "Reevaluating Amdahl's law". In: *Commun. ACM* 31.5 (May 1988), pp. 532–533. ISSN: 0001-0782. DOI: 10.1145/42411.42415. URL: https://doi.org/10.1145/42411.42415.

[3] Mark D. Hill and Michael R. Marty. "Amdahl's Law in the Multicore Era". In: *Computer* 41.7 (2008), pp. 33–38. DOI: 10.1109/MC.2008.209.

[4] Tongping Liu, Charlie Curtsinger, and Emery D. Berger. "Dthreads: efficient deterministic multithreading". In: *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*. SOSP '11. Cascais, Portugal: Association for Computing Machinery, 2011, pp. 327–336. ISBN: 9781450309776. DOI: 10.1145/2043556.2043587. URL: https://doi.org/10.1145/2043556.2043587.

[5] Marek Olszewski, Jason Ansel, and Saman Amarasinghe. "Kendo: efficient deterministic multithreading in software". In: *SIGPLAN Not.* 44.3 (Mar. 2009), pp. 97–108. ISSN: 0362-1340. DOI: 10.1145/1508284.1508256. URL: https://doi.org/10.1145/1508284.1508256.

[6] Tao B. Schardl, William S. Moses, and Charles E. Leiserson. "Tapir: Embedding Recursive Fork-join Parallelism into LLVM's Intermediate Representation". In: *ACM Trans. Parallel Comput.* 6.4 (Dec. 2019). ISSN: 2329-4949. DOI: 10.1145/3365655. URL: https://doi.org/10.1145/3365655.

[7] Unity Technologies. *Entity Component System concepts*. URL: https://docs.unity3d.com/Packages/com.unity.entities@1.4/manual/concepts-intro.html (visited on 11/19/2025).

[8] Unity Technologies. *Structural changes concepts*. URL: https://docs.unity3d.com/Packages/com.unity.entities@1.4/manual/concepts-structural-changes.html (visited on 11/19/2025).

[9] Unity Technologies. *Structural changes concepts*. URL: https://docs.unity3d.com/6000.0/Documentation/Manual/job-system-overview.html (visited on 04/15/2026).

[10] David Wingqvist, Filip Wickström, and Suejb Memeti. "Evaluating the performance of object-oriented and data-oriented design with multi-threading in game development". In: *2022 IEEE Games, Entertainment, Media Conference (GEM)*. 2022, pp. 1–6. DOI: 10.1109/GEM56474.2022.10017610.

[11] Qi Zhao et al. "Understanding and Reaching the Performance Limit of Schedule Tuning on Stable Synchronization Determinism". In: *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques*. PACT '22. Chicago, Illinois: Association for Computing Machinery, 2023, pp. 223–238. ISBN: 9781450398688. DOI: 10.1145/3559009.3569669. URL: https://doi.org/10.1145/3559009.3569669.